

Import+Load

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 df = pd.read_csv("/content/sample_data/dirty_cafe_sales.csv")
```

Exploration

```
1 print(df.shape)
2 print(df.dtypes)
3 print(df.info())
4 print(df.isna().sum())
5 print(df.duplicated().sum())
```

```
(10000, 8)
Transaction ID      object
Item                object
Quantity            object
Price Per Unit      object
Total Spent         object
Payment Method      object
Location            object
Transaction Date     object
dtype: object
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Transaction ID         10000 non-null object
1   Item                   9667 non-null  object
2   Quantity               9862 non-null  object
3   Price Per Unit         9821 non-null  object
4   Total Spent            9827 non-null  object
5   Payment Method         7421 non-null  object
6   Location                6735 non-null  object
7   Transaction Date       9841 non-null  object
dtypes: object(8)
memory usage: 625.1+ KB
None
Transaction ID      0
Item                333
Quantity            138
Price Per Unit      179
Total Spent         173
Payment Method      2579
Location            3265
Transaction Date     159
dtype: int64
0
```

Cleaning

```
1 cols_to_clean = ['Item', 'Payment Method', 'Location', 'Transaction Date']
2 for col in cols_to_clean:
3     df[col] = df[col].replace(['UNKNOWN', 'ERROR'], np.nan)
4
5 df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')
6 df['Price Per Unit'] = pd.to_numeric(df['Price Per Unit'], errors='coerce')
7
8 df['Quantity'] = df['Quantity'].fillna(df['Quantity'].mean())
9
10 df['Total Spent'] = df['Quantity'] * df['Price Per Unit']
11 df['Total Spent'] = pd.to_numeric(df['Total Spent'], errors='coerce')
12
13 df['Transaction Date'] = pd.to_datetime(df['Transaction Date'], errors='coerce')
14
15 df = df.drop_duplicates()
16 df = df.dropna(thresh=6)
17
18 print(df.isna().sum())
```

```
Transaction ID      0
Item                785
Quantity            0
Price Per Unit      191
Total Spent         191
Payment Method      2835
Location            3569
Transaction Date     352
dtype: int64
```

▼ Sales Metrics

```
1 total_sales = df['Total Spent'].sum()
2 avg_transaction = df['Total Spent'].mean()
3 top_order = df['Total Spent'].max()
4 lowest_order = df['Total Spent'].min()
5
6 print(f"Total Sales: {total_sales:.2f}")
7 print(f"Average Transaction: {avg_transaction:.2f}")
8 print(f"Top Order: {top_order}")
9 print(f"Lowest Order: {lowest_order}")
```

```
Total Sales: 82,900.80
Average Transaction: 8.93
Top Order: 25.0
Lowest Order: 1.0
```

▼ Items

```
1 top_item = df['Item'].value_counts().head(1)
2 lowest_item = df['Item'].value_counts().tail(1)
3 sales_by_item = df.groupby('Item')['Total Spent'].sum().sort_values(ascending=False)
4 avg_item_price = df['Price Per Unit'].mean()
5
6 print(top_item)
7 print(lowest_item)
8 print(sales_by_item)
```

```
Item
Coffee    1132
Name: count, dtype: int64
Item
Tea       1037
Name: count, dtype: int64
Item
Salad      16247.258166
Sandwich   12926.831215
Smoothie   12568.781851
Juice       9943.013339
Cake       9825.671778
Coffee     6771.358681
Tea        4623.634755
Cookie     3040.423170
Name: Total Spent, dtype: float64
```

▼ Payment_Method

```
1 best_payment_method = df['Payment Method'].value_counts().head(1)
2 sales_by_payment = df.groupby('Payment Method')['Total Spent'].sum()
3 avg_by_payment = df.groupby('Payment Method')['Total Spent'].mean()
4
5 print(best_payment_method)
6 print(sales_by_payment)
7 print(avg_by_payment)
```

```
Payment Method
Digital Wallet    2233
Name: count, dtype: int64
Payment Method
Cash              19348.939397
Credit Card      19220.713581
Digital Wallet    19415.280958
Name: Total Spent, dtype: float64
Payment Method
Cash              9.054253
Credit Card      8.977447
```

```
Digital Wallet      8.943013
Name: Total Spent, dtype: float64
```

Location

```
1 sales_by_location = df.groupby('Location')['Total Spent'].sum()
2 top_location = sales_by_location.idxmax()
3
4 print(sales_by_location)
5 print(top_location)
```

```
Location
In-store    25792.385359
Takeaway    25039.956517
Name: Total Spent, dtype: float64
In-store
```

Time

```
1 sales_by_date = df.groupby('Transaction Date')['Total Spent'].sum()
2
3 months = df['Transaction Date'].dt.month_name()
4 sales_by_month = df.groupby(months)['Total Spent'].sum()
5
6 month_order = ['January', 'February', 'March', 'April', 'May', 'June',
7               'July', 'August', 'September', 'October', 'November', 'December']
8 sales_by_month = sales_by_month.reindex(month_order)
9
10 print(sales_by_month)
```

```
Transaction Date
January         6745.157546
February        6244.061706
March           6666.275181
April           6844.627193
May             6473.429839
June           7011.901376
July            6523.463974
August          6687.175559
September       6475.031352
October         6783.702132
November        6607.634755
December        6643.045583
Name: Total Spent, dtype: float64
```

Quantity

```
1 avg_quantity = df['Quantity'].mean()
2 quantity_by_item = df.groupby('Item')['Quantity'].sum().sort_values(ascending=False)
3
4 print(avg_quantity)
5 print(quantity_by_item)
```

```
3.0278576090236853
Item
Coffee      3465.679340
Juice       3378.337780
Cake        3355.309316
Salad       3321.451633
Sandwich    3315.707804
Smoothie    3210.195463
Tea         3156.451633
Cookie      3126.423170
Name: Quantity, dtype: float64
```

Correlation

```
1 corr_qty_spent = df[['Quantity', 'Total Spent']].corr()
2 corr_price_spent = df[['Price Per Unit', 'Total Spent']].corr()
3
4 print(corr_qty_spent)
5 print(corr_price_spent)
```

	Quantity	Total Spent
Quantity	1.000000	0.696616
Total Spent	0.696616	1.000000

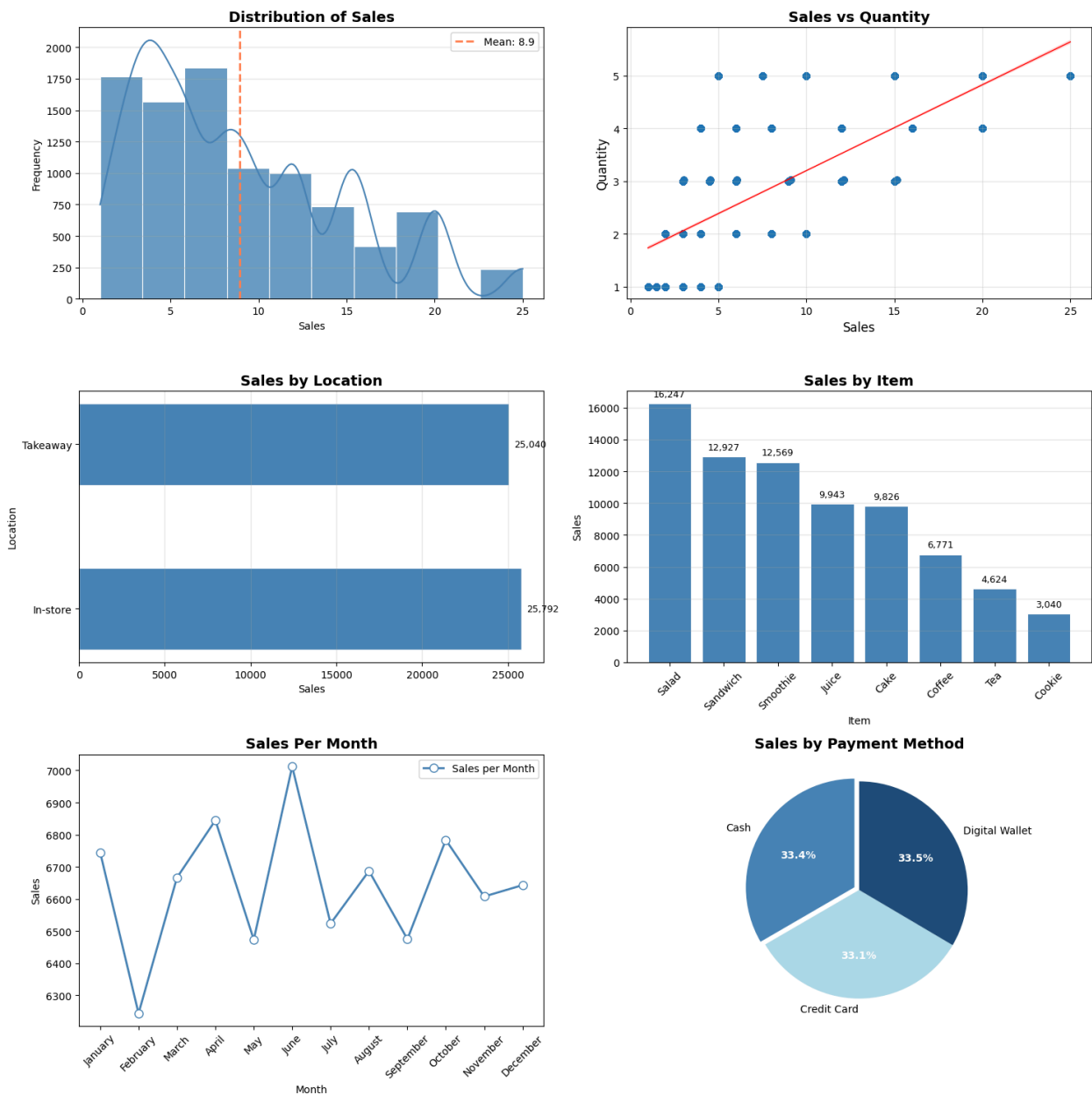
	Price Per Unit	Total Spent
Price Per Unit	1.000000	0.655859
Total Spent	0.655859	1.000000

Dashboard

```

1 fig, ax = plt.subplots(3, 2, figsize=(15, 15))
2 ax1, ax2, ax3, ax4, ax5, ax6 = ax[0,0], ax[0,1], ax[1,0], ax[1,1], ax[2,0], ax[2,1]
3
4 sns.histplot(data=df, x='Total Spent', bins=10, kde=True,
5             color='steelblue', edgecolor='white', linewidth=0.8, alpha=0.8, ax=ax1)
6 ax1.axvline(df['Total Spent'].mean(), color='coral', linewidth=2, linestyle='--',
7            label=f"Mean: {df['Total Spent'].mean():.1f}")
8 ax1.set_title('Distribution of Sales', fontsize=14, fontweight='bold')
9 ax1.set_xlabel('Sales'); ax1.set_ylabel('Frequency')
10 ax1.grid(axis="y", alpha=0.3); ax1.legend()
11
12 sns.regplot(data=df, x='Total Spent', y='Quantity', ax=ax2,
13            scatter_kws={'alpha': 0.4}, line_kws={'color': 'red', 'linewidth': 1})
14 ax2.set_title('Sales vs Quantity', fontsize=14, fontweight='bold')
15 ax2.set_xlabel('Sales', fontsize=12); ax2.set_ylabel('Quantity', fontsize=12)
16 ax2.grid(True, alpha=0.3)
17
18 bars = ax3.barh(sales_by_location.index, sales_by_location.values,
19               color='steelblue', edgecolor='white', height=0.5)
20 ax3.set_title('Sales by Location', fontsize=14, fontweight='bold')
21 ax3.set_xlabel('Sales'); ax3.set_ylabel('Location')
22 ax3.grid(axis='x', alpha=0.3)
23 for bar in bars:
24     width = bar.get_width()
25     ax3.text(width + 300, bar.get_y() + bar.get_height()/2, f"{width:,.0f}",
26            ha='left', va='center', fontsize=9)
27
28 bars = ax4.bar(sales_by_item.index, sales_by_item.values,
29               color='steelblue', edgecolor='white', linewidth=0.8)
30 ax4.set_title('Sales by Item', fontsize=14, fontweight="bold")
31 ax4.set_xlabel('Item'); ax4.set_ylabel('Sales')
32 ax4.grid(axis='y', alpha=0.3)
33 ax4.tick_params(axis='x', rotation=45)
34 for bar in bars:
35     height = bar.get_height()
36     ax4.text(bar.get_x() + bar.get_width()/2, height + 300, f"{height:,.0f}",
37            ha="center", va="bottom", fontsize=9)
38
39 ax5.plot(sales_by_month.index, sales_by_month.values, color='steelblue', linewidth=2,
40         linestyle="-", marker='o', markersize=8, markerfacecolor='white',
41         markeredgcolor='steelblue', label='Sales per Month')
42 ax5.set_title('Sales Per Month', fontsize=14, fontweight='bold')
43 ax5.set_xlabel('Month'); ax5.set_ylabel('Sales')
44 ax5.tick_params(axis='x', rotation=45)
45 ax5.legend()
46
47 wedges, texts, autotexts = ax6.pie(
48     sales_by_payment.values, labels=sales_by_payment.index,
49     autopct='%1.1f%%', startangle=90,
50     colors=["steelblue", "#ADD8E6", "#1F4E79"], explode=[0.05, 0, 0])
51 for autotext in autotexts:
52     autotext.set_fontsize(10); autotext.set_fontweight('bold'); autotext.set_color('white')
53 ax6.set_title('Sales by Payment Method', fontsize=14, fontweight='bold')
54 plt.tight_layout()
55 plt.savefig("dash.pdf", bbox_inches='tight')
56 plt.show()

```



Key Insights

```
1 insights = f""  
2 KEY INSIGHTS - Cafe Sales Analysis  
3 -----  
4 1. Total revenue across {len(df)} clean transactions: {total_sales:,.2f}  
5 2. Average transaction value: {avg_transaction:.2f}  
6 3. Best-selling item by volume: {top_item.index[0]} ({top_item.values[0]} orders)  
7 4. Top revenue-generating item: {sales_by_item.index[0]} ({sales_by_item.values[0]:,.0f})  
8 5. In-store vs Takeaway sales are nearly balanced ({sales_by_location.to_dict()})  
9 6. Moderate positive correlation (r={corr_qty_spent.iloc[0,1]:.2f}) between  
10 Quantity and Total Spent -> customers buying more units spend proportionally more.  
11 7. Payment methods are evenly split (~33% each across Cash, Credit Card, Digital Wallet),  
12 suggesting no dominant payment preference among customers.  
13 ""  
14 print(insights)
```

KEY INSIGHTS - Cafe Sales Analysis

-
1. Total revenue across 9475 clean transactions: 82,900.80
 2. Average transaction value: 8.93
 3. Best-selling item by volume: Coffee (1132 orders)
 4. Top revenue-generating item: Salad (16,247)
 5. In-store vs Takeaway sales are nearly balanced ({'In-store': 25792.38535868081, 'Takeaway': 25039.956517172566})
 6. Moderate positive correlation (r=0.70) between
Quantity and Total Spent -> customers buying more units spend proportionally more.
 7. Payment methods are evenly split (~33% each across Cash, Credit Card, Digital Wallet),
suggesting no dominant payment preference among customers.